

- **Objective:** recover linux from unexpected shutdown
- **Stumble upon:** r/archlinux

1 [A] Update grub parameters

- **Goal(s):** update grub boot parameters to splash `screen` and hide grub `menu`
 - Add `splash` parameter to kernel parameter: `GRUB_CMDLINE_LINUX_DEFAULT="... splash"`
 - Add `hidden` parameter to `GRUB_TIMEOUT_STYLE`: `GRUB_TIMEOUT_STYLE=hidden`

2 [A] Enable zswap

- **Goal(s):** enable system zswap on boot
 - Add `zswap.enable=1` to kernel parameter: `GRUB_CMDLINE_LINUX="... zswap.enabled=1"`

3 [A] Config `pacman`

- **Goal(s):** config `pacman` to:
 - Output colors: `Colors`
 - have a funny animation `IloveCandy`

3.1 Reverse Packages Upgrade

- To reverse an `official` package use Archlinux Archive @Archwiki
- To reverse an `AUR` package check out AUR Downgrade @AUR

4 [A] Fix screen tearing

- **Objective:** *Rock archlinux without a compositor*
 - **References:** Intel TearFree @Orgmode

5 [A] Screen Time Event

- **OBJECTIVE:** Do not turn off screen when a media is playing `archlinux`

-
- **[NOTE]** taken on [2025-08-14 16:24]:
 - Suspending screen saving when a media is playing via: Disable DPMS if ALSA is currently playing sound @Arch-forum
 - Adopt a `cron-job` for `dpms`:

- Script to detect playing media and set timer for display off timer

```
#!/bin/bash
# shellcheck disable=SC2155
set -euo pipefail

declare -ir _timeout=600
declare -ir _current="$(xset -display :0.0 q | awk '/Standby:/ {
↪ print $2 }')"

_enable() {
if [[ "${_current}" == "0" ]]
then
xset -display :0.0 dpms ${_timeout} ${_timeout} ${_timeout}
fi
}

_disable() {
if [[ "${_current}" != "0" ]]
then
xset -display :0.0 dpms 0 0 0
fi
}

if grep -q "RUNNING" /proc/asound/card*/pcm*/sub*/status
```

```
then
  _disable
else
  _enable
- Systemd timer and service

[Unit]
Description=Timer to check for display screen off

[Timer]
OnBootSec=5
OnUnitActiveSec=1min
Unit=dpms.service

[Install]
WantedBy=timers.target

[Unit]
Description=A dmps checker for media files

[Service]
Type=oneshot
ExecStart=/usr/local/bin/dpms-media
```

- **Reference:** Do Not Turn Display Off When a Media is Playing @Arch-forum

6 [A] Disable fallback initramfs generation

Generation of fall `initramfs` can lead to full `boot` drive and thus unable to upgrade system or packages. To disable to generation of fallback image [fn:disable-fallback-img]:

- Change `PRESETS('default' 'fallback')` line to `PRESETS=('default')` in the respective `.preset` files in `/etc/mkinitcpio.d/`
- Remove the fallback initramfs images in `/boot/`
- Update your boot loader configuration: `mkinitcpio -P`



Disabling all fallback initramfs generation will leads to booting problems when the default image fails. Make sure you have to bootable installation medium for rescue purposes

Reference:* USB flash installation medium

7 [A] User Service Proxy

Sometimes we want a `user-service` to run along with `system-services` on `systemd` init system. There are a few ways to tackle this problem.

- Elevating service privileges
- Create a proxy service that runs when a system-service started and signal the `user-service` to run on demand

```
[Unit]
Description=Start user ac.target
After=powerup.service

[Service]
Type=oneshot
ExecStart=/sbin/systemctl --user --machine=%i@ start ac.target

[Install]
WantedBy=ac.target
```

- To start the proxy-service run:

```
systemctl enable --now ac@whammou.service
```

8 [A] Mounting USB Storage

- Auto-mounting with `udisks` - this is the easiest and most frequent used method
 - **Installation:** Install the `udisks2` or `udisks2-btrfs` module via `pacman`
 - **Usage:** To use mount a USB drive first define the block device:
 - * To lists all system mount points, run: `$ lsblk`
 - * To mount a drive, run: `$ udisksctl mount -b /dev/sdb1`
 - * or to unmount: `$ udisksctl unmount -b /dev/sdb2`
- Mounting using `yazi` :
 - Install plugin `mount.yazi` via `$ ya pkg add yazi-rs/plugins:mount`
 - Create custom keybinds:

```
[[mgr.prepend_keymap]]  
on = "M"  
run = "plugin mount"
```

- Launch the *mount-manager* inside `yazi` with `M`

9 [A] System Swap and Swappiness

What are swap and swappiness and how to increase system performance by adjusting swappiness value

`Linux` divides its physical `RAM` into chunks of memory called pages. `swapping` is a the process whereby a page of memory is copied to the preconfigured space on the hard disk (*a swap page*), to free up that page of memory



The amount of `physical memory` and `swap space` is the amount of virtual memory available

- **goto:** Archlinux System Tweaks @Org

9.1 Swap Space & Swap Partition & Swap File

A `swap space` can take the form of a disk partition or a file. `Swap space` can be used for 2 purposes:

- To extend the `virtual memory` beyond the installed `physical memory`
- And also add Suspend-to-disk support @Org
 - To check swap status, use:
`swapon --show`
 - To show whole `virtual memory` as `swap` usage:
`free -h`

9.1.1 Swap Partition

A `swap partition` is a dedicated disk partition used for `swap space`. To enable the device for paging

```
swapon /dev/sdxy
```

A brief comparison between `swap partition` and `swap file`: Swap partition vs Swap file
@Opencode-session

9.1.2 Swap File

As an alternative to create an entire `partition`

- A `swap-file` offers the ability to vary its size *on-the-fly*
- And is more easily removed altogether
- This is desirable if the disk space is at premium (e.g. SSD)

1. Swap File Creation



There are some limitations of the implementation in 'btrfs' and linux swap sub-system

- Swap files on file systems spanning multiple devices are not supported
- Data profile must be single
- 'snapshot' cannot be create for sub-volumes containing active 'swap-file'
- 'swap-files' must be pre-allocated
- **copy-on-write** must be disabled individually for 'swap-files' (**i.e., swap files must have to NOCOW attribute**)

To properly create a `swap-file` on `btrfs`

- First, create *sub-volume* as *snap-shots* cannot be created for that sub-volume afterwards:

```
btrfs subvolume create /swap
```
- Create a `4GB` swap-file `swap-file` in `/swap/`:

```
btrfs filesystem mkswapfile --size 4g --uuid clear /swap/swapfile
```
- Activate the swap-file:

```
swapon /swap/swapfile
```

- Finally, edit the `fstab` configuration to add an entry for the swap-file:

```
/swap/swapfile none swap defaults 0 0
```

2. Swap File Removal

To remove a swap-file, it must be turned off first and then can be removed:

```
swapoff /swap/swapfile
```

```
rm -f /swap/swapfile
```

And remove the relevant entry from `/etc/fstab`

9.2 System Swappiness

When 'memory' reaches a certain threshold, the kernel starts looking at active memory and seeing what it can free up

- File data can be written out to the file system
- unloaded and re-loaded later
- Other data must be written to swap before it can be unloaded

The `swappiness` parameter represents the kernel's preference for writing to swap instead of files.

- It can have a value between 0 and 200
- The default value is 60
 - A low value causes the kernel to prefer freeing up open files
 - a high value causes the kernel to try to use swap space

To check the current `swappiness` value:

```
sysctl vm.swappiness
```

To temporarily set the `swappiness` value:

```
sysctl -w vm.swappiness=20
```

To set the `swappiness` value permanently, create a `sysctl.d(5)` configuration file:

```
#/etc/sysctl.d/99-swappiness.conf
```

```
vm.swappiness = 20
```

To have the `boot-loader` set the swappiness when loading the kernel

- Add a `kernel-parameter` e.g. `sysctl.vm.swappiness=20`

10 [A] Disable Internet on Sleep



There are different sleep method supported on Power Management methods @Arch-wiki

Currently using `deep` method to ensure internet is turned off on `sleep`

10.1 Advertise Internet Service

To advertise that internet service is available (*unavailable*), use NetworkManager `dispatch-script` file: `/etc/NetworkManager/dispatcher.d/` to run a script on interface state is up/down.

```
#!/usr/bin/env bash
```

```
DEVICE=${1}
STATE=${2}
check_internet() {
    # Ping gateway first (faster), then external IP if needed
    # ping -c 1 -W 2 "$IP4_GATEWAY" &>/dev/null ||
    # ping -c 1 -W 3 8.8.8.8 &>/dev/null
    ping -c 1 -W 2 google.com &>/dev/null
}
if [ "$DEVICE" = "wlan0" ]; then
    if [ "$STATE" = "up" ]; then
        if check_internet; then
            systemctl --user --machine=whammou@.host start internet.target
        else
            exit 0
        fi
    elif [ "$STATE" = "down" ]; then
        systemctl --user --machine=whammou@.host stop internet.target
    fi
fi
```



Under certain conditions NetworkManager will fail to turn off gracefully especially when running in a long session

To workaround this problem - advertise `internet.target` unit in systemd `sleep` targets e.g `suspend`, `sleep`, `suspend-then-hibernate`

```
[Service]
```

```
ExecStartPre=/usr/sbin/systemctl --user --machine=whammou@.host stop
```

```
↳ internet.target
```